

Identificador de Razas Caninas con Inteligencia Artificial

Informe Final del Proyecto — Segundo Bimestre

Asignatura: Fundamentos de Inteligencia Artificial

Docente: Ing. Vanessa Guevara, MSc.

Período académico: 2026-A

Integrantes: Alejandro Fabara · Marcos _____

Repositorio: github.com/alejoturip/Proyecto-Final-FUNDAMENTOS-DE-INTELIGENCIA-ARTIFICIAL

Resumen. Se desarrolló una aplicación web (backend y frontend) que identifica la raza de un perro a partir de una fotografía usando una red neuronal convolucional. El modelo se construyó con **Transfer Learning** sobre **MobileNetV2** y reconoce **25 razas**, con una **exactitud del 91.8 %** sobre 851 imágenes de validación que la red nunca vio durante el entrenamiento. El sistema permite entrenar el modelo con parámetros configurables, exponerlo mediante una API REST, mostrar las métricas del entrenamiento y recibir nuevas imágenes para predecir su raza en tiempo real.

Palabras clave: inteligencia artificial, clasificación de imágenes, redes neuronales, Transfer Learning, MobileNetV2, FastAPI, React.

1. Introducción

Reconocer la raza de un perro a simple vista requiere experiencia: existen más de 340 razas y muchas comparten rasgos físicos. Este proyecto aplica **visión por computadora** para automatizar esa tarea. El usuario sube una foto y el sistema responde con la raza más probable, su nivel de confianza, las tres candidatas más probables y una ficha informativa de la raza detectada. El proyecto corresponde al enfoque de **clasificación de imágenes**, uno de los propuestos en las indicaciones de la asignatura.

2. Motivación

- **¿Por qué este tema?** La clasificación de imágenes es uno de los problemas más representativos de la IA moderna y permite abordar todos los conceptos del curso (redes neuronales, entrenamiento, sobreajuste, métricas) sobre un caso concreto y visual.
- **¿Qué problema aborda?** Identificar razas es útil en veterinarias, refugios y aplicaciones de adopción, donde muchas veces llega un animal sin información de su raza.
- **¿Por qué es relevante?** Demuestra cómo, con Transfer Learning, se resuelve en una laptop un problema que hace una década exigía supercomputadoras y millones de imágenes.
- **¿Cuál es el aporte de la IA?** Extrae automáticamente los patrones visuales (orejas, hocico, pelaje, proporciones) que distinguen una raza de otra, sin programarlos a mano.

3. Objetivos

Objetivo general

Desarrollar una aplicación funcional (backend y frontend) que aplique un modelo de Inteligencia Artificial para clasificar la raza de un perro a partir de una imagen.

Objetivos específicos

- Construir y entrenar un modelo de clasificación de imágenes con Transfer Learning.
- Exponer el modelo mediante una API REST e integrarlo con una interfaz web.
- Permitir configurar los parámetros del entrenamiento y mostrar sus métricas.
- Desplegar la aplicación para que sea accesible en línea.

- Preparar la defensa técnica del proyecto.

4. Alcance

Incluye: clasificación de 25 razas caninas; interfaz web para subir imágenes y ver resultados; API con endpoints de predicción, métricas y estado; despliegue en la nube (Netlify y Render).

No incluye: detección de varios perros en una misma foto; razas fuera de las 25 entrenadas (ante una raza desconocida el modelo devuelve la más parecida, con baja confianza); identificación de perros mestizos como tales.

5. Estado del arte

- **ImageNet** (Deng et al., 2009): base de 1.2 millones de imágenes que hizo posible entrenar redes profundas de propósito general. Nuestro modelo parte de sus pesos.
- **MobileNetV2** (Sandler et al., 2018): arquitectura convolucional eficiente para dispositivos con pocos recursos; base de nuestro modelo por su equilibrio precisión/tamaño.
- **Stanford Dogs Dataset** (Khosla et al., 2011): dataset de 120 razas y 20 580 imágenes derivado de ImageNet; fuente principal de nuestras imágenes.
- **Transfer Learning** (Pan & Yang, 2010): reutilizar el conocimiento de un modelo para otra tarea con muchos menos datos; pilar metodológico del proyecto.

6. Arquitectura del sistema

Son **dos aplicaciones independientes** que se comunican por una URL: la interfaz en React (Netlify) y la API en Python (Render). La lógica de IA está separada del framework web, de modo que puede probarse y optimizarse por su cuenta.

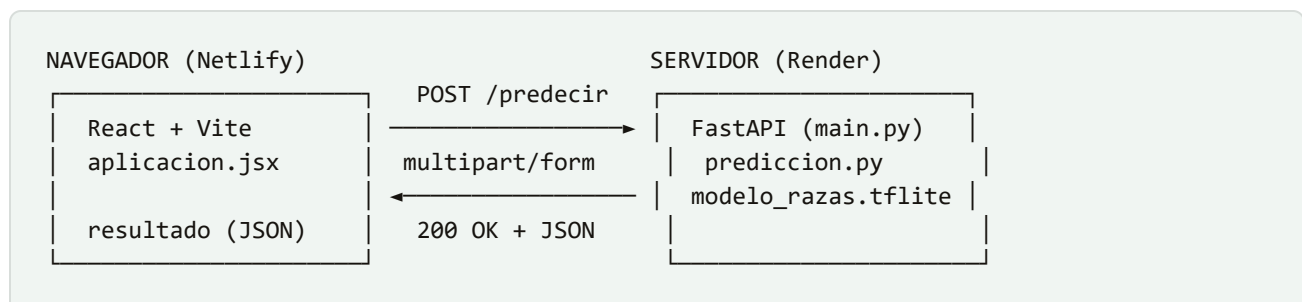


Tabla 1. Responsabilidad de cada archivo del backend.

Archivo	Responsabilidad
<code>main.py</code>	Única capa que conoce HTTP: define las rutas de la API y CORS.
<code>prediccion.py</code>	La IA en uso: recibe bytes de imagen y devuelve predicciones. No sabe de HTTP.
<code>entrenamiento.py</code>	Entrena el modelo (se ejecuta una vez en la laptop).
<code>evaluar.py</code>	Calcula las métricas de calidad del modelo.
<code>convertir_tflite.py</code>	Convierte el modelo a TensorFlow Lite para producción.
<code>razas.json</code>	"Base de datos" con la ficha de cada raza (sin motor de BD).

Flujo completo: el usuario elige una foto → React la muestra y la envía por `POST /predecir` como `multipart/form-data` → FastAPI valida tipo y tamaño → la imagen se convierte en un tensor 224×224 → el modelo devuelve 25 probabilidades → se toman las 3 mayores y se arma el JSON → React pinta raza, confianza, Top 3 y ficha.

[Captura de pantalla 1]

Pantalla principal de la aplicación con una predicción (raza, confianza y Top 3).

7. El modelo de Inteligencia Artificial

7.1 Transfer Learning con MobileNetV2

En lugar de entrenar desde cero (que requeriría millones de imágenes), se parte de MobileNetV2 preentrenada en ImageNet, que ya sabe detectar bordes, texturas, orejas y pelaje. Solo se le enseña la parte final: distinguir las 25 razas.

7.2 Entrenamiento en dos etapas

- **Etapas 1, base congelada** (10 épocas, *learning rate* $1e-3$): se entrena solo la capa de clasificación nueva; los pesos de MobileNetV2 no se tocan.
- **Etapas 2, ajuste fino** (hasta 12 épocas, *learning rate* $1e-5$, últimas 60 capas descongeladas): se afina la parte alta de la red con un *learning rate* 100 veces menor para no destruir lo aprendido. Un *EarlyStopping* detiene el proceso cuando la validación deja de mejorar.

7.3 Defensas contra el sobreajuste (overfitting)

Aumento de datos (giros, rotaciones, zoom y contraste aleatorios), *dropout* del 30 %, un conjunto de validación del 20 % que la red nunca usa para aprender, y *EarlyStopping*.

7.4 Parámetros configurables (requisito de la rúbrica)

El entrenamiento no está fijo en el código; se controla por línea de comando:

```
python entrenamiento.py --epocas-ajuste 15 --batch 16 --capas 80 --lr-ajuste 1e-5
```

Tabla 2. Parámetros configurables del entrenamiento.

Parámetro	Qué controla
--epocas-cabeza / --epocas-ajuste	Vueltas al dataset en cada etapa.
--batch	Tamaño de lote (imágenes por paso de ajuste).
--lr-cabeza / --lr-ajuste	Tasa de aprendizaje de cada etapa.
--capas	Cuántas capas finales de MobileNetV2 se descongelan.

8. Dataset

- 25 razas, de 150 a 250 imágenes por raza, **4 256 imágenes** en total.
- **21 razas** provienen de Stanford Dogs (descarga y filtrado automáticos).
- **4 razas** (dachshund, bulldog inglés, pastor australiano, akita) no existen en Stanford Dogs y se complementaron desde **Wikimedia Commons**.
- Reparto: 80 % entrenamiento / 20 % validación.

9. Resultados

Exactitud global (accuracy): 91.8 % sobre 851 imágenes de validación.

Tabla 3. Métricas promedio (macro) del modelo.

Métrica	Valor
Precision	0.917
Recall	0.913

Métrica	Valor
F1-score	0.913

Mejores razas (F1): Pastor Alemán 1.000, Caniche 0.986, Yorkshire 0.975, Beagle 0.974, San Bernardo 0.969. **Razas más difíciles (F1):** Dachshund 0.746, Akita 0.809, Chihuahua 0.828, Pastor Australiano 0.833.

Las razas con menor rendimiento son, en su mayoría, las complementadas desde Wikimedia Commons, cuyas fotos son más heterogéneas que las curadas de Stanford Dogs. Esto confirma un principio central del curso: **la calidad de los datos condiciona directamente la del modelo.**

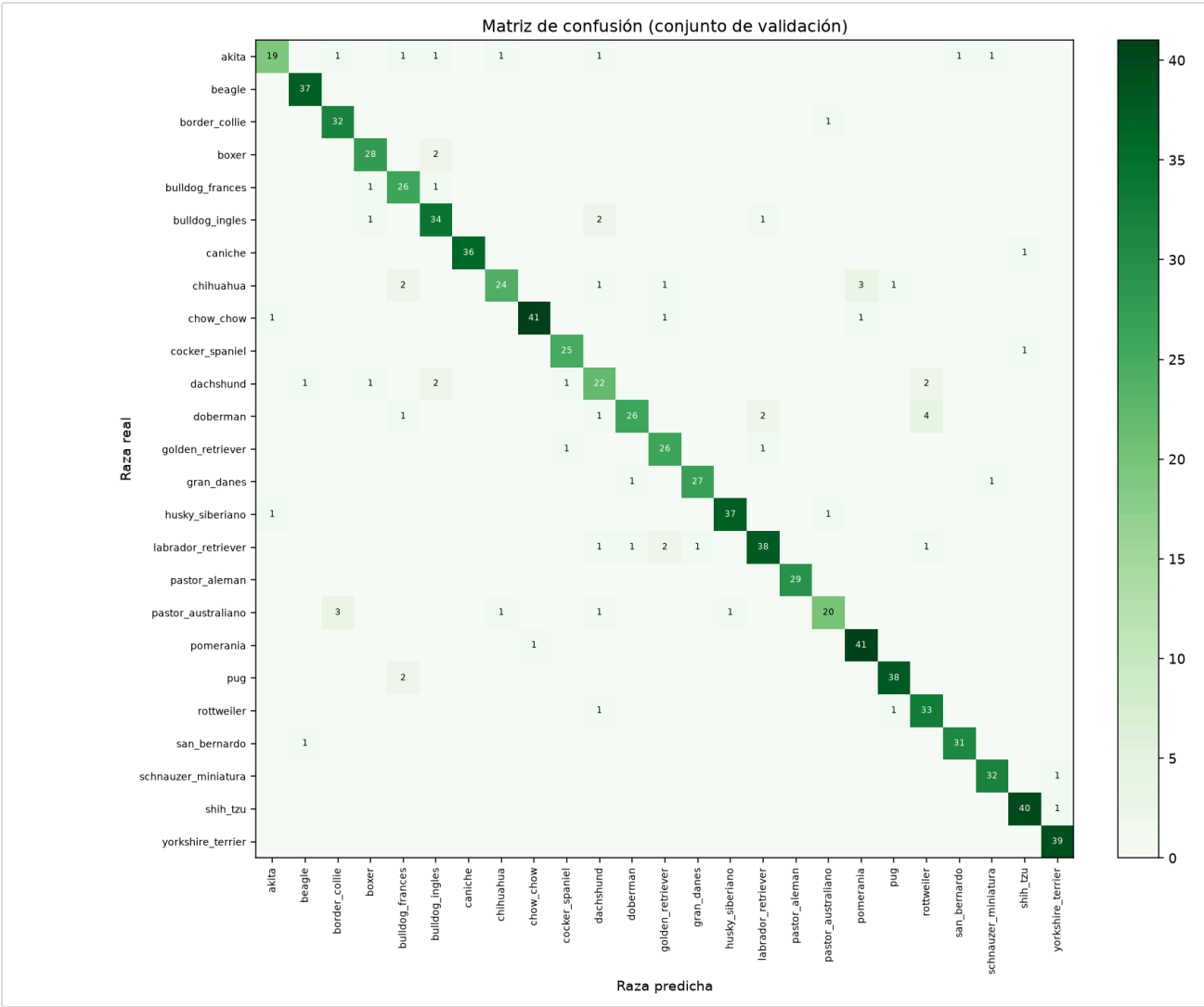


Figura 1. Matriz de confusión (validación). La diagonal representa los aciertos.

10. Optimización para producción (TensorFlow Lite)

El plan gratuito de Render ofrece 512 MB de RAM y TensorFlow completo (~450 MB) no entra: el servicio se caía por falta de memoria. La solución fue convertir el modelo a TensorFlow Lite con cuantización dinámica, verificando que las predicciones fueran idénticas a las del original.

Tabla 4. Impacto de la conversión a TensorFlow Lite.

	Antes (Keras + TF)	Después (TFLite)
Tamaño del modelo	23 MB	2.4 MB
RAM en el servidor	~450 MB	~150 MB
Runtime instalado	TensorFlow (~400 MB)	ai-edge-litert (pocos MB)



11. Manual técnico (instalación y uso)

Requisito: Python 3.12 (TensorFlow no soporta 3.13/3.14) y Node.js 20+.

Backend:

```
cd backend
py -3.12 -m venv venv
venv\Scripts\activate
pip install -r requirements-desarrollo.txt # TensorFlow para entrenar y correr el modelo
uvicorn main:app --reload # API en http://localhost:8000
```

Frontend:

```
cd frontend
npm install
copy .env.example .env
npm run dev # App en http://localhost:5173
```

Reentrenar todo el pipeline: `python ejecutar_todo.py`

Despliegue: frontend en Netlify (`npm run build` con `VITE_API_URL` apuntando a Render) y backend en Render (root `backend` , `PYTHON_VERSION=3.12.10` , start `uvicorn main:app --host 0.0.0.0 --port $PORT`).

12. Cumplimiento de los requisitos mínimos

Tabla 5. Cumplimiento de los requisitos de la rúbrica.

Requisito	Cómo se cumple
Entrenar el modelo	<code>entrenamiento.py</code> (Transfer Learning en 2 etapas).
Crear un API e implementarlo	FastAPI (<code>main.py</code>), desplegado en Render.
Configurar parámetros	Parámetros por línea de comando en <code>entrenamiento.py</code> .
Mostrar métricas del entrenamiento	Endpoint <code>/metricas</code> y panel en la app.
Recibir nuevos datos y predecir	<code>POST /predecir</code> (subir foto → predicción).

13. Conclusiones

- Con Transfer Learning se alcanzó 91.8 % de exactitud en 25 razas usando ~4 000 imágenes y una laptop sin GPU, algo inviable entrenando desde cero.
- La arquitectura desacoplada permitió probar, optimizar y desplegar cada parte por separado.
- La conversión a TensorFlow Lite fue clave para desplegar en un servidor con recursos limitados, reduciendo el consumo de memoria unas tres veces.
- El rendimiento por raza demostró de forma empírica que la calidad del dataset es tan importante como la arquitectura del modelo.

14. Recomendaciones y trabajos futuros

- Mejorar las razas débiles ampliando y depurando sus imágenes (sobre todo las tomadas de Wikimedia Commons).
- Detección previa de perro: rechazar imágenes que no contengan un perro antes de clasificar.
- Ampliar el número de razas y añadir manejo explícito de mestizos.
- Cuantización completa a enteros (int8) para acelerar aún más la inferencia.
- Aplicación móvil: el formato TFLite permite llevar el modelo a Android/iOS sin servidor.

Referencias

- Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., & Fei-Fei, L. (2009). ImageNet: A large-scale hierarchical image database. *IEEE Conference on Computer Vision and Pattern Recognition*.
- Khosla, A., Jayadevaprakash, N., Yao, B., & Fei-Fei, L. (2011). Novel dataset for fine-grained image categorization: Stanford Dogs. *CVPR Workshop on Fine-Grained Visual Categorization*.
- Pan, S. J., & Yang, Q. (2010). A survey on transfer learning. *IEEE Transactions on Knowledge and Data Engineering*, 22(10), 1345-1359.
- Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). MobileNetV2: Inverted residuals and linear bottlenecks. *IEEE Conference on Computer Vision and Pattern Recognition*.